

Pixel Flow Langevin

First Author	Second Author
Affiliation	Affiliation
email@domain	email@domain

1 Pixel Flow

The following discussion only considers only one scale, but can be generalized to multiple scales using modified A operators.

1.1 Final-stage construction with shared noise

We specialize PixelFlow to the last (full-resolution) stage, where the end-point is at the same scale as x_1 . Let $G \triangleq \text{Up} \circ \text{Down}(\cdot, 2)$ denote downsampling by a factor of 2 followed by upsampling back to full resolution. Let $a \triangleq t_0^0$ and $c \triangleq t_1^0$ be the (stage-dependent) mixing coefficients. Assume the stage start and end share the same Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$.

Define the stage endpoints

$$x_{\text{start}} = a G x_1 + (1 - a) \varepsilon, \quad (1)$$

$$x_{\text{end}} = c x_1 + (1 - c) \varepsilon. \quad (2)$$

For within-stage time $\tau \in [0, 1]$, the PixelFlow interpolation state is

$$x_\tau = (1 - \tau) x_{\text{start}} + \tau x_{\text{end}}. \quad (3)$$

Substituting (1)–(2) into (3) yields a linear-Gaussian form

$$x_\tau = H_\tau x_1 + \sigma_\tau \varepsilon, \quad (4)$$

where

$$H_\tau \triangleq (1 - \tau) a G + \tau c I, \quad (5)$$

$$\sigma_\tau \triangleq (1 - \tau)(1 - a) + \tau(1 - c). \quad (6)$$

1.1.1 From velocity to endpoint estimates

For a linear interpolation path $x_\tau = (1 - \tau) x_{\text{start}} + \tau x_{\text{end}}$, the (within-stage) velocity is constant and equals $v = x_{\text{end}} - x_{\text{start}}$. PixelFlow trains $\mu_\theta(x_\tau, \tau)$ to approximate this velocity. Thus we form the *endpoint reconstructions*

$$\hat{x}_{\text{start}}(x_\tau, \tau) \triangleq x_\tau - \tau \mu_\theta(x_\tau, \tau), \quad \hat{x}_{\text{end}}(x_\tau, \tau) \triangleq x_\tau + (1 - \tau) \mu_\theta(x_\tau, \tau). \quad (7)$$

These satisfy $\hat{x}_{\text{start}} \approx x_{\text{start}}$ and $\hat{x}_{\text{end}} \approx x_{\text{end}}$ when the velocity prediction is accurate.

1.1.2 Estimating the clean endpoint x_1 (a denoiser) from reconstructed endpoints

Because the final-stage endpoints share the same noise ε , the pair $(x_{\text{start}}, x_{\text{end}})$ provides redundant linear information about x_1 and ε . A stable way to extract a denoised estimate of x_1 is to solve a regularized weighted least squares problem:

$$\hat{x}_1 \triangleq \arg \min_x \frac{1}{2(1-a)^2} \|\hat{x}_{\text{start}} - aGx\|^2 + \frac{1}{2(1-c)^2} \|\hat{x}_{\text{end}} - cx\|^2 + \frac{\lambda_x}{2} \|x\|^2, \quad (8)$$

where $\lambda_x \geq 0$ is a small ridge parameter to stabilize the inversion (especially since G is not invertible).

The normal equations for (8) are

$$\left(\frac{a^2}{(1-a)^2} G^\top G + \frac{c^2}{(1-c)^2} I + \lambda_x I \right) \hat{x}_1 = \frac{a}{(1-a)^2} G^\top \hat{x}_{\text{start}} + \frac{c}{(1-c)^2} \hat{x}_{\text{end}}. \quad (9)$$

This is an SPD linear system and can be solved efficiently with CG. Once $\hat{x}_1$ is available, we may also recover the shared noise by

$$\hat{\varepsilon} \triangleq \frac{\hat{x}_{\text{end}} - c\hat{x}_1}{1-c} \approx \varepsilon, \quad (10)$$

or equivalently $\hat{\varepsilon} = (x_\tau - H_\tau \hat{x}_1) / \sigma_\tau$.

A cheaper approximation is to ignore the impact of $\hat{x}_{\text{start}}$:

$$\boxed{\hat{x}_1 \approx \hat{x}_{\text{end}} / c} \quad (11)$$

1.1.3 Score of the marginal $p_{\text{PF}}(x_\tau, \tau)$ via a generalized Tweedie identity

We note that

$$x_\tau = H_\tau x_1 + \sigma_\tau \varepsilon, \quad (12)$$

which implies that it is the Gaussian corrupted version of $H_\tau x_1$. Then a generalized Tweedie identity for the linear channel implies

$$\nabla_{x_\tau} \log p_{\text{PF}}(x_\tau, \tau) = \frac{1}{\sigma_\tau^2} \left(H_\tau \mathbb{E}[x_1 | x_\tau, \tau] - x_\tau \right). \quad (13)$$

Replacing the conditional mean by $\hat{x}_1$ yields the practical score estimator

$$\boxed{\hat{s}_{\text{PF}}(x_\tau, \tau) \triangleq \frac{1}{\sigma_\tau^2} \left(H_\tau \hat{x}_1(x_\tau, \tau) - x_\tau \right)}. \quad (14)$$

Equivalently, using $\hat{\varepsilon} = (x_\tau - H_\tau \hat{x}_1)/\sigma_\tau$,

$$\hat{s}_{\text{PF}}(x_\tau, \tau) = -\frac{1}{\sigma_\tau} \hat{\varepsilon}(x_\tau, \tau). \quad (15)$$

When we make the approximation in (11), we obtain

$$\hat{s}_{\text{PF}}(x_\tau, \tau) = -\frac{1}{\sigma_\tau} \hat{\varepsilon}(x_\tau, \tau) = \frac{1}{\sigma_\tau^2} (x_\tau - H_\tau \hat{x}_1) \quad (16)$$

Implementation summary. Given (x_τ, τ) : (i) compute $\hat{x}_{\text{start}}, \hat{x}_{\text{end}}$ via (7); (ii) solve (9) by CG to obtain $\hat{x}_1$; (iii) compute $\hat{s}_{\text{PF}}(x_\tau, \tau)$ via (14). This $\hat{s}_{\text{PF}}$ can be substituted for s_{PF} in (25)–(26). This involves a CG-Solve.

Remark (cost and reuse of CG infrastructure). The score computation introduces an additional SPD solve (9). In practice, this solve can reuse the same CG codepath and preconditioning infrastructure used for the inverse-problem solves (e.g., diagonal or FFT-based preconditioners when applicable), and can be run for a small, fixed number of CG iterations as an inexact denoiser without materially degrading sampling performance.

2 Inverse problem: setting

2.1 Inverse problem likelihood

We consider a linear inverse problem

$$b = Ax_1 + n, \quad n \sim \mathcal{N}(0, \eta^2 I), \quad (17)$$

which implies the Gaussian likelihood

$$p(b \mid x_1) \propto \exp\left(-\frac{1}{2\eta^2} \|Ax_1 - b\|^2\right). \quad (18)$$

2.2 PixelFlow prior on x_τ and constrained posterior

Let $p_{\text{PF}}(x_\tau, \tau)$ denote the (implicit) PixelFlow density at time τ on the state x_τ , and define its score

$$s_{\text{PF}}(x_\tau, \tau) \triangleq \nabla_{x_\tau} \log p_{\text{PF}}(x_\tau, \tau). \quad (19)$$

We place the standard normal prior on the shared noise,

$$p(\varepsilon) = \mathcal{N}(\varepsilon; 0, I). \quad (20)$$

Introduce the deterministic constraint (4) via a Dirac delta:

$$p(x_\tau \mid x_1, \varepsilon) = \delta(x_\tau - (H_\tau x_1 + \sigma_\tau \varepsilon)). \quad (21)$$

Then the joint posterior over $(x_1, \varepsilon, x_\tau)$ is

$$\pi(x_1, \varepsilon, x_\tau | b) \propto p(b | x_1) p_{\text{PF}}(x_\tau, \tau) p(\varepsilon) \delta(x_\tau - (H_\tau x_1 + \sigma_\tau \varepsilon)). \quad (22)$$

Marginalizing x_τ yields the posterior on (x_1, ε) :

$$\boxed{\pi(x_1, \varepsilon | b) \propto p(b | x_1) p_{\text{PF}}(H_\tau x_1 + \sigma_\tau \varepsilon, \tau) p(\varepsilon).} \quad (23)$$

We will use Gibbs sampling to sampling from the above distribution. In particular, we will update x_1 and ε alternatively.

2.3 Gradients for Langevin updates

Let $x_\tau = H_\tau x_1 + \sigma_\tau \varepsilon$. The log-posterior (up to constants) is

$$\log \pi(x_1, \varepsilon | b) = -\frac{1}{2\eta^2} \|Ax_1 - b\|^2 + \log p_{\text{PF}}(x_\tau, \tau) - \frac{1}{2} \|\varepsilon\|^2. \quad (24)$$

Differentiating gives

$$g_{x_1}(x_1, \varepsilon) \triangleq \nabla_{x_1} \log \pi(x_1, \varepsilon | b) = \frac{1}{\eta^2} A^\top (b - Ax_1) + H_\tau^\top s_{\text{PF}}(x_\tau, \tau), \quad (25)$$

$$g_\varepsilon(x_1, \varepsilon) \triangleq \nabla_\varepsilon \log \pi(x_1, \varepsilon | b) = -\varepsilon + \sigma_\tau s_{\text{PF}}(x_\tau, \tau). \quad (26)$$

Note that we are updating both x_1 and ε , and the update of ε is based on the score so that we get a good estimate.

2.4 Single-CG preconditioned Langevin with stochastic RHS

We update (x_1, ε) by a preconditioned unadjusted Langevin (pULA) step. Choose stepsize $h > 0$ and define an SPD “precision preconditioner”

$$P^{-1} \triangleq \frac{1}{\eta^2} A^\top A + \lambda I, \quad \lambda > 0. \quad (27)$$

The x_1 increment is chosen to have mean $(h/2)P g_{x_1}$ and covariance hP . Using a stochastic RHS factorization, note that

$$P^{-1} = LL^\top, \quad L \triangleq \begin{bmatrix} \frac{1}{\eta} A^\top & \sqrt{\lambda} I \end{bmatrix}, \quad (28)$$

since $LL^\top = \frac{1}{\eta^2} A^\top A + \lambda I$.

Thus, if $\xi_1 \sim \mathcal{N}(0, I_m)$ and $\xi_2 \sim \mathcal{N}(0, I_n)$ are independent, then

$$\Delta x_1 \text{ solves } P^{-1} \Delta x_1 = \frac{h}{2} g_{x_1}(x_1, \varepsilon) + \sqrt{h} \left(\frac{1}{\eta} A^\top \xi_1 + \sqrt{\lambda} \xi_2 \right), \quad (29)$$

implies $\Delta x_1 \sim \mathcal{N}(\frac{h}{2} P g_{x_1}, hP)$. We compute (29) using conjugate gradients (CG), requiring one CG solve per iteration.

We update ε by an (unpreconditioned) Langevin step:

$$\varepsilon^+ = \varepsilon + \frac{h}{2} g_\varepsilon(x_1, \varepsilon) + \sqrt{h} \xi_3, \quad \xi_3 \sim \mathcal{N}(0, I_n). \quad (30)$$